

Workshop 4

Performance Scorecard and Shipping

Honest numbers for your project, and a repo a stranger can run.

Travis Sim

NUS · Singapore

Thu 4 Feb 2027 · COM3/COM4

Where we are

Today's question: could a stranger trust your numbers and run your project?

DONE

Build night 4

Thu 28 Jan

Backtest validation and failure testing

TODAY

Workshop 4

Thu 4 Feb

Performance scorecard and shipping

NEXT

Build night 5

Thu 11 Feb

Demo rehearsal

Demo Day: Thu 18 Feb

Today

18:00 to 20:00. A third talk, a third demos and lab, a third your own build.

18:00 — 10 min
Progress circle
where each project stands

18:10 — 40 min
Teaching
scorecard numbers, then shipping

18:50 — 25 min
Live demos
five demos, you predict first

19:15 — 15 min
Mini-lab
score the sample ledger, break a number

19:30 — 25 min
Open build
your own project, leads rotate

19:55 — 5 min
Push and close
one line on the next step

What you leave with

By 19:55, in your repo.

1

A tested commit

Integrated, and npm test passes.

2

A README

Run steps that work from a clean checkout.

3

Devnet evidence

An explorer link that resolves.

4

One known limitation

Named in the README.

5

A 3-minute demo script

With a recorded backup.

6

Peer feedback

Someone else ran it.

A ledger is the input

One row per closed trade. The scorecard does the rest.

id	strategy	side	opened	closed	entry	exit	gross USD
1	basis	short	+00:10	+00:15	108.05	108.05	+0.000
2	market-maker	short	+00:10	+01:10	108.04	107.96	+0.713
3	market-maker	short	+00:15	+01:15	108.04	107.96	+0.713
4	basis	short	+01:10	+01:40	108.01	108.01	+0.000
5	market-maker	long	+01:05	+02:05	108.01	108.09	+0.686
6	market-maker	short	+01:15	+02:15	108.01	108.08	-0.694
7	market-maker	short	+01:20	+02:20	108.01	108.08	-0.721

45 trades in the full ledger

Source: demo-repo/code/out/ledger.json (45 trades from the sample strategies, simulated on recorded data; not advice)

1. One row per closed trade
2. Gross only: the scorecard applies fees and slippage
3. Optional on-chain fields: signature, fees, status

Hit rate alone says nothing

Two made-up strategies, ten trades each.

STRATEGY A

9 of 10 trades win

each win pays +1

the one loss costs -12

net -3 per ten trades

STRATEGY B

4 of 10 trades win

each win pays +3

each of 6 losses costs -1

net +6 per ten trades

Hit rate needs the size of wins and losses beside it.

Illustrative numbers, not from our data

Win size, loss size, break-even

The hit rate you need depends on what a win pays and a loss costs.

```
break-even hit rate = average loss / (average win + average loss)
```

NO COSTS (0 + 0 bps per leg)	
Hit rate	42%
Average win / loss	+0.69 / -0.44
Break-even hit rate	39.0%
Net P&L (USD)	+3.00

FEE 5 + SLIPPAGE 2 bps per leg	
Hit rate	11%
Average win / loss	+0.45 / -1.40
Break-even hit rate	75.8%
Net P&L (USD)	-53.70

Same 45 trades. Only the costs changed.

Source: demo-repo/code/out/cost-scenarios.json (simulated on recorded data; not advice)

Gross is not net

Same 45 trades, three cost settings.

FEE 0 + SLIPPAGE 0 BPS

+3.00

net P&L, USD

gross +3.00

costs 0.00

FEE 5 + SLIPPAGE 2 BPS

-53.70

net P&L, USD

gross +3.00

costs -56.70

FEE 12 + SLIPPAGE 5 BPS

-134.60

net P&L, USD

gross +3.00

costs -137.60

The venue gap in our recorded data was typically 1 to 3 bps (largest 6.3 bps). A round trip here pays 14 bps.

Costs are paid on every leg, win or lose.

Source: demo-repo/code/out/cost-scenarios.json; venue gaps in code/recorded/SOURCES.md. Simulated, on recorded data; not advice.

Drawdown and time under water

How far it fell, and how long it stayed down.



MAX DRAWDOWN, ZERO COSTS

3.84 USD

bigger than the 3.00 USD it ends up ahead

UNDER WATER, ZERO COSTS

82%

longest stretch 29 min of 40

WITH 5 + 2 BPS COSTS

99%

under water, max drawdown 53.70 USD

Even with zero costs the equity spent most of the window below a previous peak.

Source: lab demo 2 and code/out/cost-scenarios.json. Simulated, on recorded data; not advice.

Turnover

How much you traded compared with what you hold.

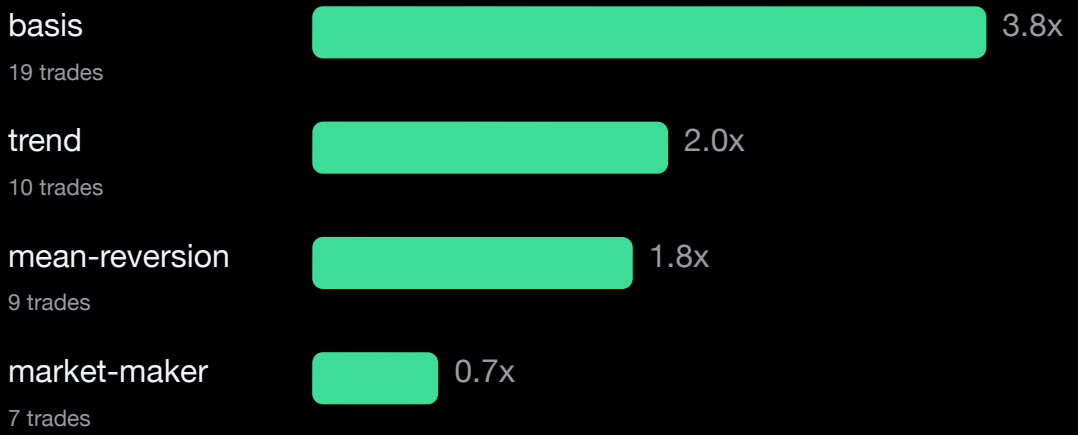
TURNOVER, ALL 45 TRADES

8.3x

83,000 USD traded on 10,000 USD of capital, in 40 minutes

two legs per trade, 1,000 USD each

PER STRATEGY



More turnover means more costs. Check it before you celebrate a gross number.

Source: demo-repo/code/out/scorecard.json (capital 10,000 USD, 40 minutes). Simulated, on recorded data; not advice.

The scorecard on one page

Eight numbers, one ledger.



1 Net, not gross

2 Hit rate against break-even

3 Drawdown and time under water

Source: lab demo 1 (recorded data). Simulated; not advice.

The code behind it

Plain functions, no framework.

src/scorecard.ts

```
export function netUsd(t: LedgerTrade, c: CostSettings): number {
  const bps = c.feeBps + (t.paysSlippage ? c.slippageBps : 0);
  return t.grossUsd - t.legs * (bps / 10_000) * t.notionalUsd;
}

export function breakEvenHitRate(win: number, loss: number): number {
  return loss / (win + loss);
}

export function failureProbability(hit: number, breakEven: number, n: number): number {
  const se = Math.sqrt((hit * (1 - hit)) / n);
  return se === 0 ? (hit < breakEven ? 1 : 0) : normCdf((breakEven - hit) / se);
}
```

1. Net = gross minus costs on every leg
2. Break-even comes from the two average payoffs
3. A normal approximation, with a warning under 30 trades

Source: demo-repo/code/src/scorecard.ts (excerpt)

Run it

One command, recorded data, no network.

```
$ npm run demo -- --recorded
```

Strategy	Trades	Hit	AvgWin	AvgLoss	Payoff	NetUSD	MaxDD	Under%	Turn x	PFail
basis	19	0%	n/a	−1.33	n/a	−25.29	25.29	99%	3.8	n/a
market-maker	7	43%	+0.20	−1.02	0.20	−3.45	4.06	94%	0.7	98%
mean-reversion	9	0%	n/a	−1.29	n/a	−11.57	11.57	80%	1.8	n/a
trend	10	20%	+0.81	−1.88	0.43	−13.39	13.50	73%	2.0	100%
ALL (one ledger)	45	11%	+0.45	−1.40	0.32	−53.70	53.70	99%	8.3	100%

```
One ledger, in words
  45 trades; 11.1% won after costs; the break-even hit rate for these payoffs is 75.8%.
  Gross +3.00 USD, costs −56.70, net −53.70. Costs ate 1892% of the gross profit.
What this cannot say: 40 minutes of one quiet night is one path, not a distribution. A good number here is a reason to test more, never a result.
```

1 Table per strategy, then the whole ledger

2 Names the limit: 40 minutes, one path

Source: terminal output of the sample code (assets/terminal-demo.txt), recorded data of 21 Oct 2026

Probability of failure

Is the hit rate you saw the real one?

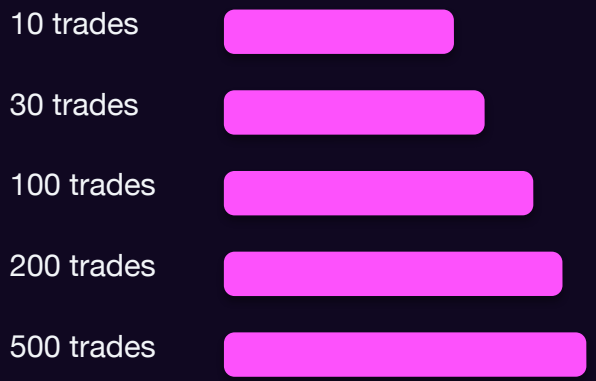
THE IDEA

You saw a hit rate over N trades. The real one is unknown. Break-even says how low it can go before you lose. The chance it is below break-even shrinks as N grows.

Same hit rate, more trades, surer answer.

HIT RATE 45%, BREAK-EVEN 50%

chance the real hit rate is below break-even



Our own ledger has 45 trades.

Source: demo-repo/code/out/failure-table.json (hit rate 45%, wins pay 1x losses, break-even 50%). Normal approximation; illustrative.

What simulated numbers can and cannot say

Say it on the dashboard, not in the Q&A.

THEY CAN SAY

- Whether an idea is worth testing more
- Which costs matter most
- How deep and how long a drawdown looked on this data

THEY CANNOT SAY

- What happens next
- A return you could earn
- Busy markets, or your own order's impact

Our sample: 40 minutes on one quiet night (21 Oct 2026, 17:14 to 17:54 UTC). Simulated, on recorded data; not advice.

What Solana itself costs

Real capture: 2,680 transactions in five finalized blocks.

BASE FEE

5,000

lamports per signature, fixed by the protocol

MEDIAN FEE PAID

5,222

lamports = 0.0006 USD = 0.006 bps of a 1,000 USD trade

PAID A PRIORITY FEE

66%

of transactions; the 90th percentile paid 10,000 lamports in total priority

FAILED

35.3%

of 2,680 transactions; 28% to 44% per block

Put fees, priority fees, Jito tips and the failed rate in your dashboard when you have real transactions. Jito tips: not captured.

Source: demo-repo/code/recorded/solana-costs.json, public mainnet RPC, 21 Oct 2026 19:53 UTC. Five blocks, one evening: a sample, not a measurement of Solana.

What judges look at

Utility, tests, devnet evidence.

1

Utility

Does it solve a real problem for someone?

2

Tests

Can a stranger run npm test and see it pass?

3

Devnet evidence

An explorer link that resolves.

Never simulated returns.

README anatomy

Six blocks. A first-time reader sees the benefit and the limit.

1	What it is	One sentence, one reader.
2	Run it	Exact commands from a clean checkout.
3	What you will see	One picture or three lines.
4	Devnet evidence	An explorer link, or "none".
5	Known limitation	One honest sentence.
6	Safety	Devnet only, no keys, simulated is labelled.

templates/README.template.md

```
# <project name>
<One sentence: what it does and for whom.>
**Status:** <works on devnet / prototype / simulation only>. R
## Run it (from a fresh checkout)
```
npm install
npm test
npm run demo -- --recorded
```
Expected output: <paste the first 5 lines you actually see>. T
## What you will see
<A screenshot or a 3-line description. Name the one number or
## Devnet evidence
<Explorer link with ?cluster=devnet for at least one real tran
## Known limitation
```

Source: demo-repo/code/templates/README.template.md (excerpt)

Tests and devnet evidence

Show it, do not describe it.

TESTS

```
$ npm test
```

One command runs them. They pass from a clean checkout. One test per number you print.

DEVNET EVIDENCE

```
https://explorer.solana.com/tx/<signature>?
cluster=devnet
```

A link is not evidence until it resolves. Never paste a key.

Evidence is a transaction you made on devnet. Not a screenshot of a chart.

Secrets and repo hygiene

Scan before you push. Run it on your own project.

```
$ npm run ship-check -- ../my-project
```

Secrets	.env, keypair files, key-shaped text
.gitignore	.env and node_modules listed
Licence	MIT is a fine default
Junk	.DS_Store, logs, big files

If a real key ever reached git, rotate it. Deleting the file is not enough.

```
$ npm run ship-check -- .

Ship check: .
  PASS  README at the top of the project
  PASS  README says how to install and run
  PASS  README names one known limitation
  PASS  README says devnet only, no real funds
  PASS  Tests exist
  PASS  One command runs the tests
  PASS  No .env or keypair files in the project
  PASS  No keys or secrets inside files
  PASS  .gitignore covers .env and node_modules
  PASS  Licence file
  PASS  Demo script or pitch notes
  WARN  Devnet evidence (explorer link or signature)
        no devnet explorer link (.../tx/<signature>?cluster=devnet)
  PASS  Recorded backup for the demo
  PASS  No junk or oversized files
NEARLY READY: 13/14 checks pass, 0 fail, 1 warn
```

Source: terminal output of the ship check on the sample code itself (assets/terminal-shipcheck.txt)

Demo script

Three minutes. Say it twice with a timer.

0:20

Problem

Who loses what, with one example.

1:00

Run

The exact command. Say what to look at first.

0:40

Evidence

A passing test run or an explorer link.

0:30

Limit

One thing it does not do.

0:20

Next

One concrete next step.

```
Template: templates/DEMO-SCRIPT.template.md
```

The 2-minute pitch

Five parts. Write one sentence for each.



1 Problem

0 to 20 s

Who loses what, and why.

2 What it does

20 to 50 s

One thing. Show it.

3 Evidence

50 to 80 s

Tests pass. Devnet link. One output.

4 Limit

80 to 105 s

It does not X, so Y.

5 Ask

105 to 120 s

Run it. Next I will Z.

`Template: templates/PITCH-2MIN.md`

Cold-start test

The README has to work for someone who is not you.

1

Copy

A clean folder: no node_modules, no out.

2

Follow

Only the README. Nothing from memory.

3

Watch

The peer types. You stay silent.

4

Fix

Change the README, not the peer.

```
$ npm run smoke
```

npm install, npm test and the demo, in a temp copy

Smoke test first, then a peer. Feedback sheet: templates/PEER-FEEDBACK.md

Recorded backup

Decide before you walk up. The network will fail at the worst moment.

Network fails	Run with --recorded. The data is in the repo.
Laptop fails	Screenshots and a PDF in the repo.
A command is slow	Know what you will say while it runs.

Test the backup, not just the live path.

Demo 1: scorecard from a ledger

PREDICT FIRST

At zero costs, does this ledger make money? What about at 7 bps per leg?

```
$ npm run demo -- --fee-bps 0 --slippage-bps 0
```

YOU SHOULD SEE

- net and hit rate, with costs at 0 and at 7 bps
- break-even hit rate against the hit rate
- the verdict flips as the sliders move

Fallback: Recorded data is bundled. Open lab/01-scorecard.html offline.



Try it: nus-solana-demos.vercel.app/0008/01-scorecard.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data). Simulated; not advice.

Demo 2: drawdown and time under water

PREDICT FIRST

With zero costs, how much of the 40 minutes is spent below a previous peak?

```
$ open lab/02-underwater.html
```

YOU SHOULD SEE

- red is every moment below the running peak
- max drawdown, time under water, longest stretch
- drag the cursor: minutes since the last peak

Fallback: Same recorded ledger every time.



Try it: nus-solana-demos.vercel.app/0008/02-underwater.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data). Simulated; not advice.

Demo 3: hit rate versus payout

PREDICT FIRST

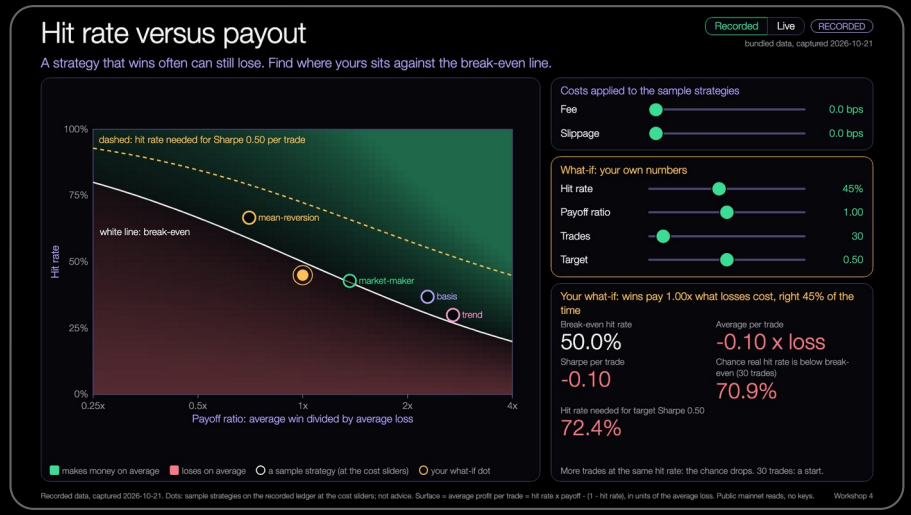
Where does a 45% hit rate with equal wins and losses land? How many trades until you trust it?

```
$ open lab/03-hit-rate-payoff.html
```

YOU SHOULD SEE

- the white line is break-even
- your what-if dot, and the chance it is below the line
- the sample strategies move with the cost sliders

Fallback: All maths runs in the page; no network needed.



Try it: nus-solana-demos.vercel.app/0008/03-hit-rate-payoff.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data). Simulated; not advice.

Demo 4: honest versus misleading dashboard

PREDICT FIRST

Which of the five tricks would have fooled you first?

```
$ open lab/04-honest-dashboard.html
```

YOU SHOULD SEE

- left: gross, annualised, best strategy only
- right: net, drawdown, all strategies
- click a row: what it hides

Fallback: Same ledger, computed by the code, no network.



Try it: nus-solana-demos.vercel.app/0008/04-honest-dashboard.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data). Simulated; not advice.

Demo 5: ship-readiness check

PREDICT FIRST

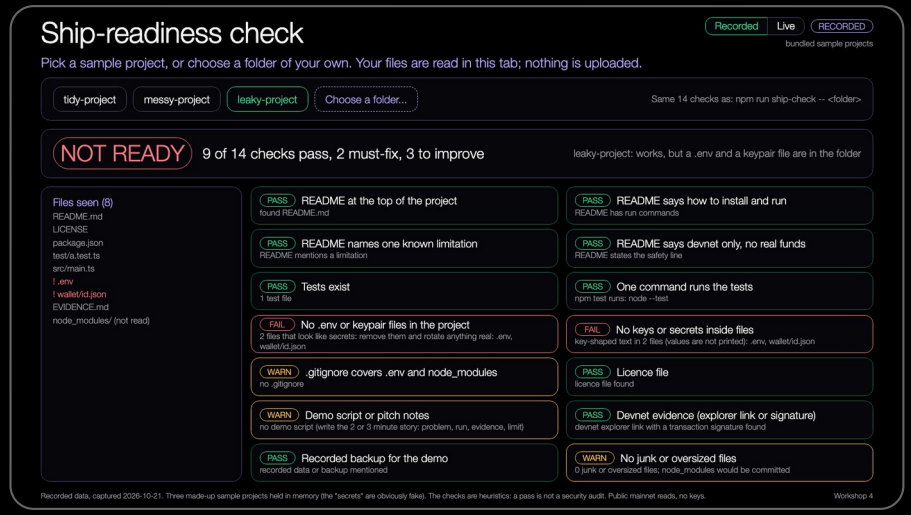
Which of the three sample projects would you ship? What stops the other two?

```
$ npm run ship-check -- ../my-project
```

YOU SHOULD SEE

- 14 checks: pass, warn or fail
- leaky-project: .env and a keypair file caught
- Live: does the devnet link resolve?

Fallback: The page has three sample projects built in.



Try it: nus-solana-demos.vercel.app/0008/05-ship-check.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

Source: demo-repo/lab/screenshots (captured from the lab demos on recorded data). Simulated; not advice.

Try the demos yourself

OPEN ON YOUR LAPTOP

`nus-solana-demos.vercel.app/0008`

Open it on your laptop and move the sliders.

Five demos: scorecard, drawdown, hit rate versus payout, honest versus misleading dashboard, ship check. Recorded data; nothing to install.



Scan to open the demos

Mini-lab

19:15 to 19:30. Fifteen minutes.

1

Run

Score the sample ledger on recorded data

2

Read

Find hit rate, break-even, drawdown

3

Break

Break one number on purpose

4

Explain

Which scorecard line caught it?

Finish line: one number broken, and the scorecard line that showed it.

Build menu

19:30 to 19:55. Pick one, or bring your own project.

BEGINNER	Run the scorecard and put three numbers in your README
BEGINNER	Finish your README from the template
STANDARD	Dashboard on your own ledger (--ledger)
STANDARD	Clean-checkout run by a peer
ADVANCED	Statistics that fit your lane, integrated
ADVANCED	A recorded-demo backup

Three takeaways

- [ ] Hit rate means nothing without payoffs, costs and drawdown.
- [ ] Say what simulated numbers cannot say, on the dashboard.
- [ ] Ship what a stranger can run: README, tests, devnet evidence, one limitation.

Branch out

After today, in your own time.

BEGINNER

Run ship-check on your project and fix every FAIL.

STANDARD

Score your own ledger with --ledger and add a drawdown chart to your README.

ADVANCED

Add a statistic that fits your lane (failed-transaction rate, fee share) and test it.

Share back

Before you leave, and by Thu 11 Feb.

TODAY

1

Push your commit, README and one known limitation.

THU 11 FEB

2

Bring your 3-minute demo script and your peer feedback.

ANY TIME

3

Send your repo link to the leads.

Get the code

REPO

`github.com/travisim/solana-agentic-trading-nus`

`folder: 0008-performance-scorecard-shipping/demo`

```
1  $ npm install
```

```
2  $ npm run demo -- --recorded
```

Run both from the demo/code folder. Node 24. No keys, no AI credits. Recorded data is bundled.



Scan for today's demos and code

Let's connect



LinkedIn

linkedin.com/in/travisim



Telegram

t.me/travisim


SOLANA